Diplomatura en programación web full stack con React JS

Módulo 6:

Administración, API Rest y despliegue

Unidad 1:

CRUD (parte 1)



Presentación

En esta unidad comenzamos con la creación del CRUD (Create, Read, Update y Delete) de novedades. Para esto vamos a crear una nueva tabla en nuestra base de datos y programaremos las funcionalidades de listar y dar de alta.



Objetivos

Que los participantes logren…

- Crear la tabla necesaria para poder manipular las novedades.
- Crear un listado de las novedades de la base de datos en el administrador.
- Implementar el código necesario para dar de alta las novedades.



Bloques temáticos

1. Listado de novedades.
2. Alta de novedades.

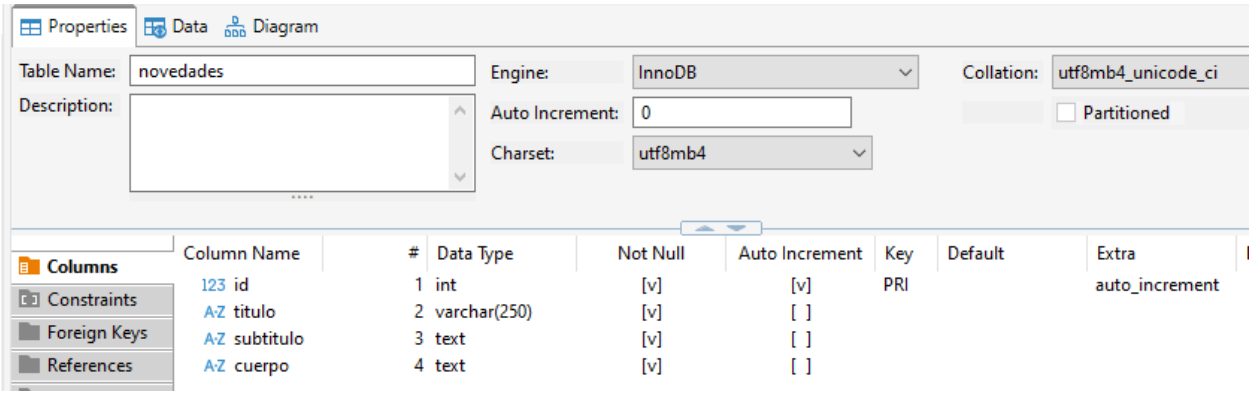
1. Listado de novedades

Paso 1

Con nuestra base de datos ya definida en la anterior unidad vamos a crear la tabla **novedades** la cual nos va a servir para listar, modificar, agregar y eliminar novedades.

La tabla contará con 4 campos:

- **id:** un entero que será nuestra clave primaria, Not Null y Auto Increment.
- **titulo:** un varchar de máximo 250 caracteres, Not Null
- **subtitulo**: un text, Not Null
- **cuerpo:** otro text, Not Null



Cuando aceptemos estos valores podemos cargar como mínimo 2 noticias a nuestra tabla para poder empezar a trabajar.

Paso 2

Crearemos el archivo **models/novedadesModel.js** donde iremos incorporando todas las funciones que nos permitirán consultar, crear, actualizar o eliminar las novedades.

La función **getNovedades** devuelve un array de filas de la tabla novedades.

```
var pool = require('./bd');

async function getNovedades() {
    var query = "select * from novedades order by id desc";
    var rows = await pool.query(query);
    return rows;
}

module.exports = { getNovedades }
```

Paso 3

En el archivo **routes/admin/novedades.js** importamos nuestro modelo de novedades y agregamos el código necesario para enviarlas como variable en el template.

```
var express = require('express');
var router = express.Router();
var novedadesModel = require('./../../models/novedadesModel');

router.get('/', async function (req, res, next) {
  var novedades = await novedadesModel.getNovedades();
 res.render('admin/novedades', {
    layout: 'admin/layout',
    usuario: req.session.nombre,
    novedades
   });
});

module.exports = router;
```

Paso 4

Modificamos también nuestro template **views/admin/novedades.hbs** para que liste las novedades recibidas desde el controlador. Para este fin, primero agregamos el código necesario para mostrar una fila estática (que no refleje datos de la base de datos), la cual podemos usar de ejemplo para ajustar nuestro diseño.

```
<div class="container" style="margin-top: 50px;">
    {{!-- fila 1 --}}
    <div class="row">
        <p class="text-end">Hola {{usuario}} ! <a class="btn btn-
danger" href="/admin/login/logout">Cerrar sesión <i class="fas fa-
sign-out-alt"></i></a></p>
    </div>
    {{!-- fin fila 1 --}}
        {{!-- fila 2 --}}
        <div class="row">
            <div class="col-7">
                <h3>Listado de Novedades</h3>
            </div>
            <div class="col-5 text-end">
                <a class="btn btn-success" href=""><i class="fa fa-
plus" aria-hidden="true"></i> Nuevo</a>
            </div>
        </div>
        {{!-- fin fila 2 --}}
```

```
{{!-- fila 3 --}}
        <div class="row">
            <div class="col">
                <table class="table">
                    <thead>
                        <tr class="table-dark">
                            <th scope="col">#</th>
                            <th scope="col">Titulo</th>
                            <th class="text-end">Acciones</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr>
                            <th scope="row">1</th>
                            <td>titulo</td>
                            <td class="text-end">
                                <a class="btn btn-secondary" href="">
                <i class="fa fa-pencil" aria-hidden="true"></i></a>
                                <a class="btn btn-secondary" href="">
                <i class="fa fa-trash" aria-hidden="true"></i></a>
                            </td>
                        </tr>
                    </tbody>
                </table>
            </div>
        </div>
    {{!-- fin fila 3 --}}
```

Paso 5

Finalmente en el archivo **view/admin/novedades.hbs** agregaremos el bucle each de Handlebars para recorrer el array de novedades que recibimos.

```
<tbody>
    {{#each novedades}}
    <tr>
        <th scope="row">{{id}}</th>
        <td>{{titulo}}</td>
        <td class="text-center">
            <a href="#" title="Editar" class="btn btn-sm btn-secondary">
            <i class="fa fa-pencil"></i></a>
            <a href="#" title="Eliminar" class="btn btn-sm btn-secondary">
            <i class="fa fa-trash"></i></a>
        </td>
    </tr>
  {{/each}}
</tbody>
```

2. Alta de novedades

Paso 1

En el archivo **models/novedadesModel.js** agregaremos la función necesaria para dar de alta una novedad.

```
async function insertNovedad(obj) {
    try {
        var query = "insert into novedades set ? ";
        var rows = await pool.query(query, [obj]);
        return rows;
    } catch (error) {
        console.log(error);
        throw error;
    }
}

module.exports = { getNovedades, deleteNovedadById,
insertNovedad }
```

Paso 2

Modificamos nuestro archivo **views/admin/novedades.hbs** creamos un link a una nueva ruta que utilizaremos para mostrar el formulario donde ingresaremos las novedades.

```
<div class="col-2 text-right">
    <a href="/admin/novedades/agregar" class="btn btn-success">
    <i class="fa fa-plus"></i>Nuevo</a>
</div>
```

Paso 3

En el archivo **routes/admin/novedades.js** creamos el nuevo manejador de ruta que mostrará el formulario de alta.

```
router.get('/agregar', (req, res, next) => {
  res.render('admin/agregar', {
    layout: 'admin/layout'
  });
});
```

Paso 4

Creamos el archivo **views/admin/agregar.hbs** con el código necesario para el formulario. Dirigimos el action a **/admin/novedades/agregar**.

```
<div class="container" style="margin:100px auto">
    <div class="row">
        <div class="col-6 offset-3">
            <h4>Agregar una nueva novedad</h4>
            <form action="/admin/novedades/agregar" method="post" >
                <div class="mb-3 row">
                    <input type="text" class="form-control"
                           placeholder="Titulo" name="titulo">
                </div>
                <div class="mb-3 row">
                    <textarea type="text" class="form-control"
                              placeholder="Subtitulo"
                              name="subtitulo"></textarea>
                </div>
                <div class="mb-3 row">
                    <textarea type="text" class="form-control"
                              placeholder="Cuerpo" name="cuerpo">
                  </textarea>
                </div>
                <button type="submit" class="btn btn-
                 primary">Guardar</button>
            </form>
    </div>
</div>
```

Paso 5

De vuelta en el archivo **routes/admin/novedades.js**, agregaremos el controlador necesario para capturar los datos enviados por el formulario. Una vez recibidos, validamos que tengamos todos los datos necesarios y se los pasamos a la función del model para guardarlos en la base de datos.

En caso de no tener todos los datos necesarios o que se produzca un error en el grabado de la información volveremos a mostrar el mismo template pasadole una variable que indique que hubo un error y una descripción del mismo.

```
router.post('/agregar', async (req, res, next) => {
  try {
    if (req.body.titulo != "" && req.body.subtitulo != "" &&
        req.body.cuerpo != "") {
      await novedadesModel.insertNovedad(req.body);
      res.redirect('/admin/novedades')
    } else {
      res.render('admin/agregar', {
        layout: 'admin/layout',
        error: true, message: 'Todos los campos son requeridos'
      })
    }
  } catch (error) {
    console.log(error)
    res.render('admin/agregar', {
      layout: 'admin/layout',
      error: true, message: 'No se cargo la novedad'
    });
  }
});
```

Paso 6

En el archivo **view/admin/agregar.hbs** agregamos el **if** para que imprima el error en caso de haber uno.

```
{{#if error}}
    <h5>{{message}}</h5>
{{/if}}
```

Conclusión

En resumen, aplicando los conceptos aprendidos comenzamos con la construcción del módulo de administración para nuestro proyecto.

En la próxima unidad veremos…

Cómo seguir integrando los conocimientos aprendidos para incorporar más funcionalidades a nuestro módulo de administración.



Bibliografía utilizada y sugerida

Artículos de revista en formato electrónico:

Handlebars. (s.f.). Handlebars.js: Minimal templating on steroids. https://handlebarsjs.com/

npm. (s.f.). npmjs.com: Node Package Manager. https://www.npmjs.com/